lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 01b — Entrées-sorties et allocation dynamique

```
bash — R3.05
$ ./moyenne < notes.txt 2> erreurs.log
Moyenne : 13.75
$ valgrind -q ./moyenne < notes.txt
Moyenne : 13.75
```

Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

Au programme

01

Les entrées-sorties standard

stdin, stdout, stderr et redirections
Caractère par caractère : getchar, putchar
Ligne par ligne : fgets, puts (et pourquoi gets est interdit)
Avec format : printf, scanf

02

L'allocation dynamique

Les limites de l'allocation statique
La mémoire d'un processus : code, données, tas, pile
malloc, calloc, realloc, free
Erreurs classiques et outils (valgrind)

Prérequis (CM 01a) : types, tableaux, pointeurs (& et *), passage de paramètres par adresse.

01

Les entrées-sorties standard

Comment un programme lit le clavier et écrit à l'écran

Trois flux standard

Trois pseudo-fichiers sont ouverts automatiquement au lancement de chaque programme. Ils sont déclarés dans <stdio.h>.

0 `stdin`

Entrée standard

descripteur

Le plus souvent associée au clavier.

1 `stdout`

Sortie standard

descripteur

Le plus souvent l'écran. Tamponnée : les caractères sont écrits par paquets (souvent à chaque \n).

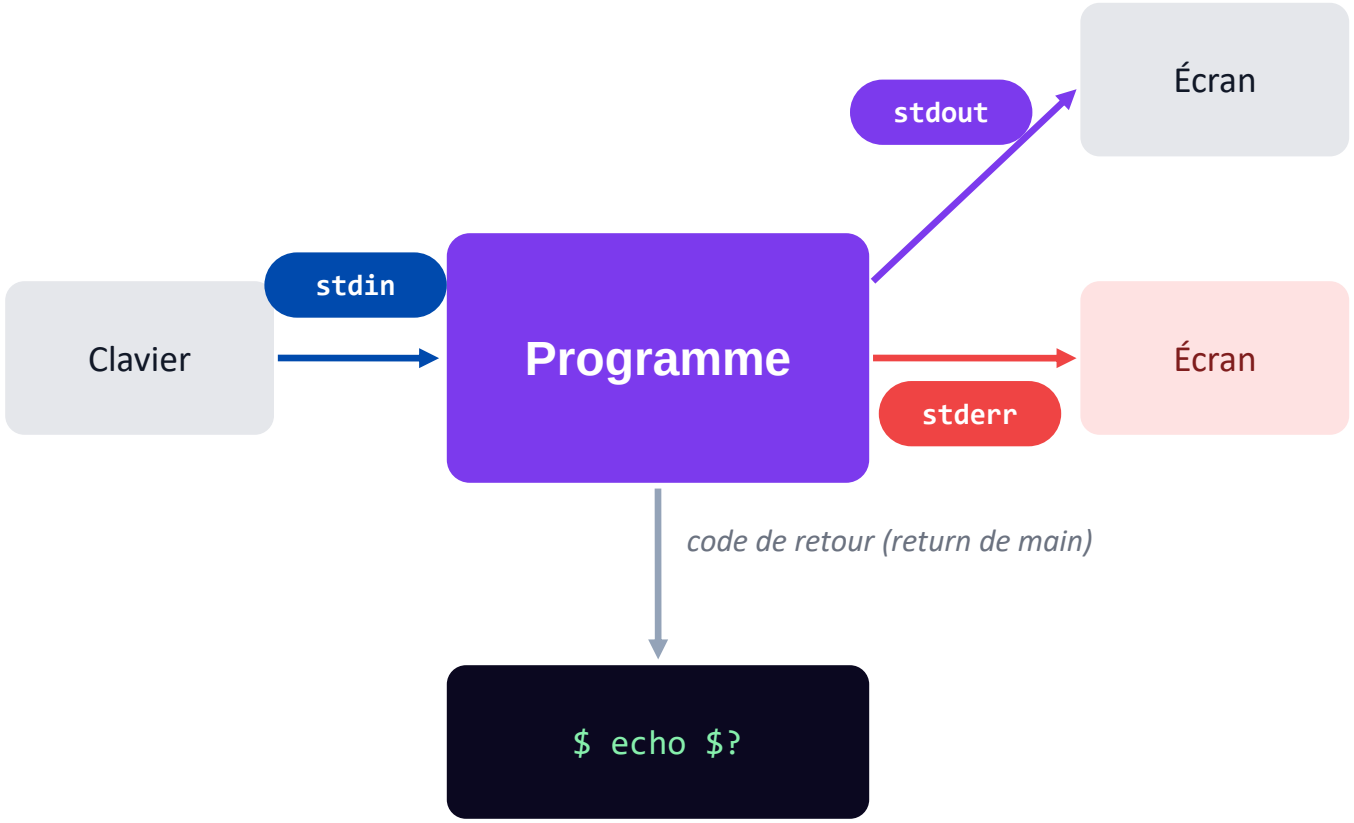
2 `stderr`

Sortie d'erreur

descripteur

L'écran elle aussi, mais non tamponnée : chaque message est écrit immédiatement.

Un programme et ses flux



Redirections du shell

`prog < entree.txt`
stdin lit le fichier

`prog > sortie.txt`
stdout va dans le fichier

`prog 2> erreurs.txt`
stderr va dans le fichier

`prog1 | prog2`
stdout de prog1 → stdin de prog2

`echo $?`
affiche le code de retour

printf ou fprintf ?

flux.c

```c
#include <stdio.h>

int main(void)
{
    printf("Résultat : 42\n");            // -> stdout
    fprintf(stderr, "Attention !\n");     // -> stderr
    return 0;
}
```

terminal

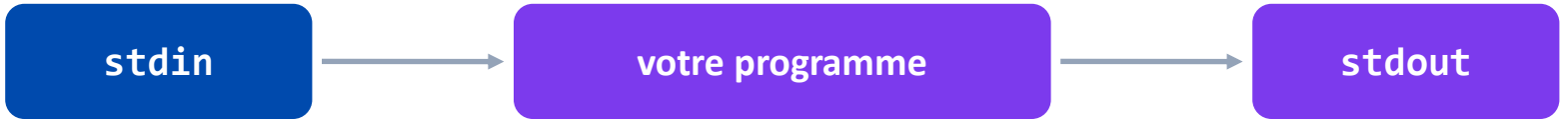
```
$ ./flux > sortie.txt
Attention !
$ cat sortie.txt
Résultat : 42
```

printf(...)

est un raccourci pour fprintf(stdout, ...). fprintf permet de choisir le flux de sortie.

Bonne pratique
Les messages d'erreur vont sur stderr : ils restent visibles même quand la sortie est redirigée vers un fichier.

Trois niveaux d'échange



01

Caractère par caractère

Lire

```
getchar()
```

Écrire

```
putchar(c)
```

Le plus simple : un octet à la fois.

02

Ligne par ligne

Lire

```
fgets(s, n, stdin)
```

Écrire

```
puts(s)
```

Une ligne entière, jusqu'au \n.

03

Avec format

Lire

```
scanf("%d", &n)
```

Écrire

```
printf("%d", n)
```

Conversion texte ↔ int, double…

Caractère par caractère

`int getchar(void);`
Lit un caractère sur stdin. Renvoie EOF en fin de fichier ou en cas d'erreur.

`int putchar(int c);`
Écrit le caractère c sur stdout. Renvoie EOF en cas d'erreur.

Pourquoi int et pas char ?
getchar doit pouvoir renvoyer les 256 caractères possibles et EOF (−1). Avec char c = getchar(); la fin de fichier peut ne jamais être détectée.

copie.c

```c
#include <stdio.h>

int main(void)
{
    int c;                  // int, pas char !
    while ((c = getchar()) != EOF)
        putchar(c);
    return 0;
}
```

terminal

```
$ ./copie
bonjour              (tapé au clavier)
bonjour
^D                   (Ctrl+D : fin de fichier)
$ ./copie < copie.c > copie2.c
```

Ligne par ligne : gets est interdit

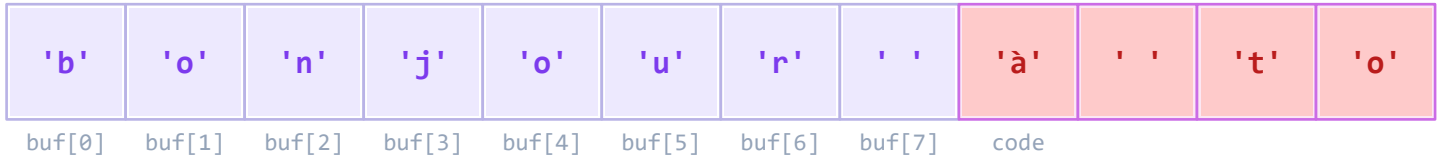
```c
char buf[8];
int  code = 1234;                    // variable voisine en mémoire

gets(buf);                           // NON : ne connaît pas la taille de buf
fgets(buf, sizeof buf, stdin);       // OUI : lit au plus 7 caractères + '\0'
```

`gets()` a été retirée du langage en C11. gcc affiche « the gets function is dangerous ».

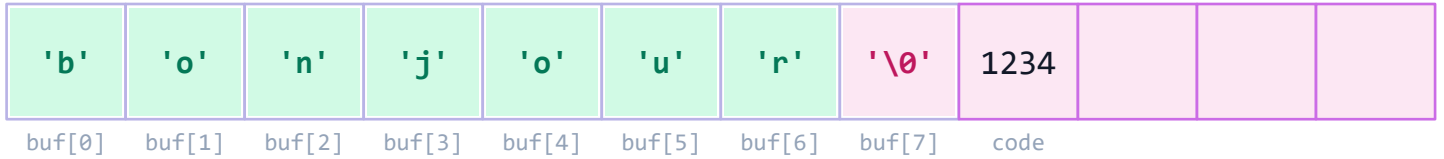
Saisie : `bonjour à tous`

avec gets



… code est écrasé : faille de débordement de tampon

avec fgets



« à tous » reste dans stdin pour la lecture suivante

Lire et écrire une ligne

`char *fgets(char *s, int n, FILE *f);`
Lit au plus n − 1 caractères. S'arrête après le \n (qui est conservé) et ajoute '\0'. Renvoie NULL en fin de fichier.

`int puts(const char *s);`
Écrit s puis un retour à la ligne sur stdout. Renvoie EOF en cas d'erreur.

fgets garde le \n et puts en ajoute un : pour recopier une ligne, utilisez fputs(s, stdout), qui n'ajoute rien.

lignes.c

```c
#include <stdio.h>

int main(void)
{
    char ligne[256];
    while (fgets(ligne, sizeof ligne, stdin) != NULL)
        fputs(ligne, stdout);
    return 0;
}
```

terminal

```
$ ./lignes < lignes.c
#include <stdio.h>
...
```

Avec format : printf et scanf

`int printf(const char *format, ...);`
Écriture formatée sur stdout.

`int scanf(const char *format, ...);`
Lecture formatée sur stdin.

largeur et précision

```c
printf("[%5d]\n", 42);        // [   42]
printf("[%-5d]\n", 42);       // [42   ]
printf("[%8.2f]\n", 3.14159); // [    3.14]
printf("%x %o\n", 255, 8);    // ff 10
```

Format	Type
%d	int
%u	unsigned int
%ld	long
%f	double (printf)
%lf	double (scanf)
%e	notation exponentielle
%c	caractère
%s	chaîne de caractères
%x	hexadécimal
%o	octal
%p	adresse (pointeur)
%zu	size_t (sizeof)
%%	le caractère %

Exemple : lire avec scanf

mult.c

```c
#include <stdio.h>

int main(void)
{
    int a, b;
    printf("Deux entiers : ");
    if (scanf("%d %d", &a, &b) != 2) {
        fprintf(stderr, "Saisie invalide\n");
        return 1;
    }
    printf("%d fois %d donne %d\n", a, b, a * b);
    return 0;
}
```

```
$ ./mult
Deux entiers : 6 7
6 fois 7 donne 42
```

```
$ ./mult
Deux entiers : six
Saisie invalide
$ echo $?
1
```

&a, &b

scanf a besoin des adresses pour modifier a et b.

!= 2

scanf renvoie le nombre de valeurs lues : toujours le vérifier.

02

L'allocation dynamique

Réserver de la mémoire pendant l'exécution

Les limites de l'allocation statique

Réserver de l'espace par une définition de variable (simple ou tableau), c'est l'allocation statique : la taille est fixée à la compilation et ne peut plus changer à l'exécution.

```
int notes[30];    // et s'il y a 31 étudiants ?
```



… 30 cases, ni plus ni moins

La solution naïve

Réserver statiquement une très grande zone (char tab[1 << 30];), puis distribuer et redistribuer soi-même des morceaux de cette zone pendant l'exécution.

Sous-utilisation

Une grande partie de la zone ne sert à rien.

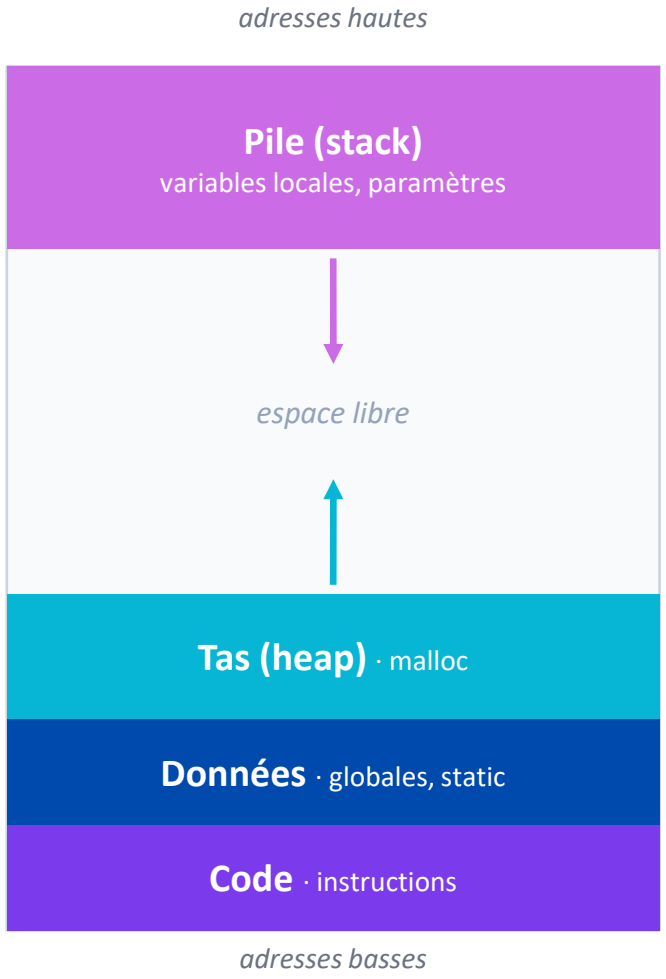
Toujours limitée

Les limites de l'allocation statique s'appliquent : un programme n'a pas accès à toute la mémoire de la machine.

Mauvaise utilisation

Il faut écrire son propre gestionnaire de mémoire : complexe et source de bugs.

La mémoire d'un processus



segments.c

```c
#include <stdlib.h>

int compteur = 0;

int main(void)
{
    int n = 10;
    int *t = malloc(n * sizeof *t);
    /* ... */
    free(t);
    return 0;
}
```

main : code

compteur : données

n et t : pile

***t (10 int) : tas**

Le tas est géré par malloc / free : les zones y vivent jusqu'à ce qu'on les libère.

Les 4 fonctions de <stdlib.h>

- L'espace mémoire n'est réservé qu'au moment de l'appel
- Sa taille peut être modifiée en cours d'exécution, sans perte de données
- Le système gère seul la mémoire réservée : il suffit de la rendre à la fin

malloc `void malloc(size_t taille);`

Réserve « taille » octets. Zone non initialisée.

calloc `void calloc(size_t nb, size_t taille);`

Réserve nb × taille octets, mis à 0.

realloc `void realloc(void p, size_t taille);`

Change la taille d'une zone ; le contenu est conservé.

free `void free(void p);`

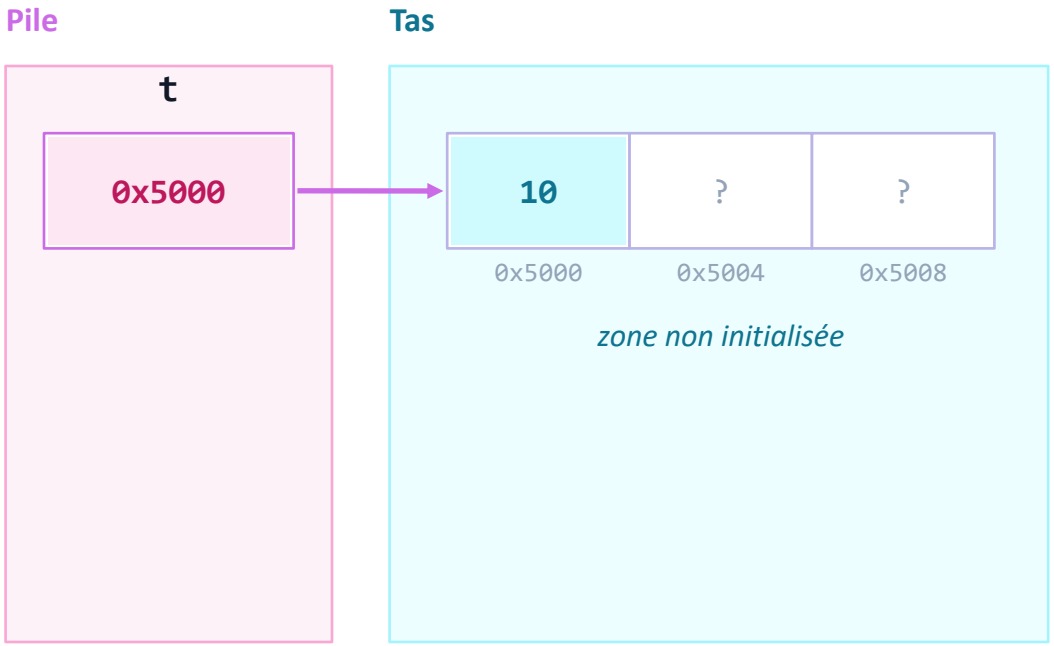
Libère la zone : elle est rendue au système.

malloc : réserver de la mémoire

malloc.c

```c
#include <stdlib.h>

int *t = malloc(3 * sizeof(int));
if (t == NULL) {            // plus de place ?
    perror("malloc");
    exit(EXIT_FAILURE);
}
t[0] = 10;
```



- Réserve taille octets dans le tas
- Renvoie l'adresse du début, ou NULL s'il n'y a plus de place : à toujours tester
- En C, le cast (int *) est inutile : void * se convertit automatiquement (il est obligatoire en C++)

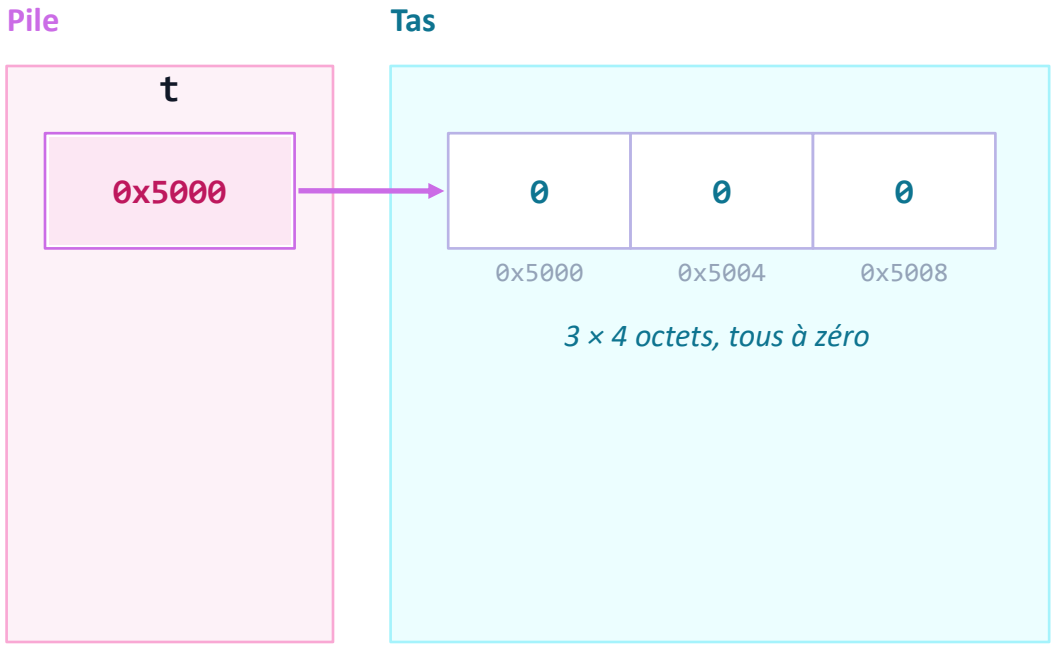
Astuce : écrivez malloc(n * sizeof *t). Si le type de t change, la taille suit automatiquement.

calloc : réserver et mettre à zéro

calloc.c

```c
int *t = calloc(3, sizeof(int));
if (t == NULL) {
    perror("calloc");
    exit(EXIT_FAILURE);
}
```

- Demande une zone de nb cases contiguës de taille octets chacune
- Renvoie l'adresse du début, ou NULL
- Toutes les cases sont initialisées à 0



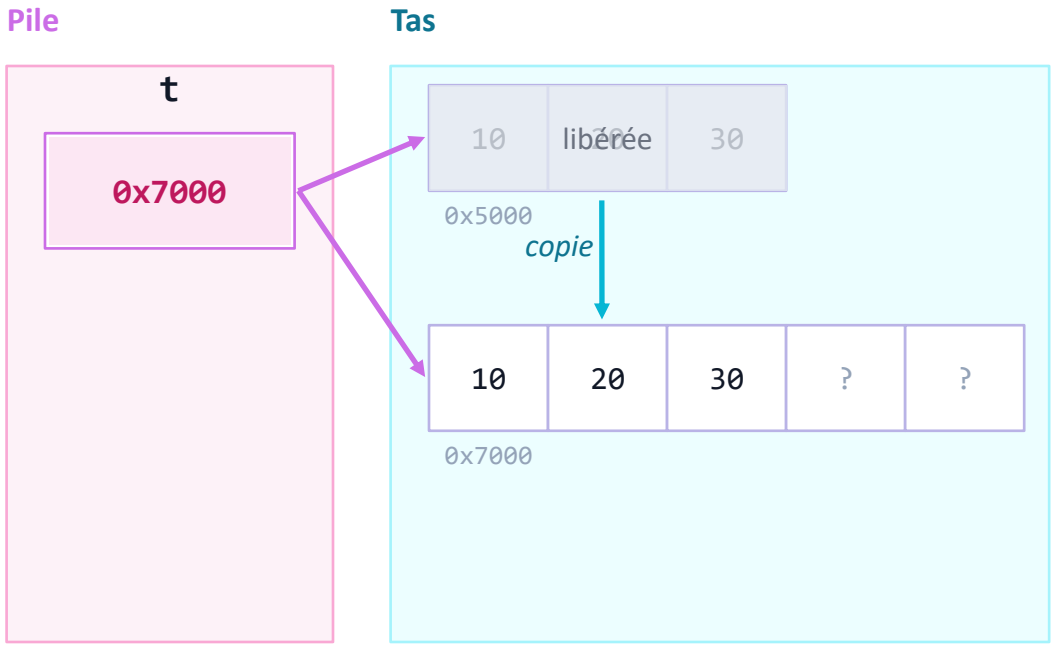
`malloc(3 * sizeof(int))`	? ? ?
`calloc(3, sizeof(int))`	0 0 0

realloc : changer la taille

realloc.c

```c
int *tmp = realloc(t, 5 * sizeof(int));
if (tmp == NULL) {      // échec : t reste valide
    free(t);
    exit(EXIT_FAILURE);
}
t = tmp;
```

- Le contenu est conservé (jusqu'à la plus petite des deux tailles)
- La zone peut être déplacée : l'adresse renvoyée peut changer
- p doit venir de malloc, calloc ou realloc (ou valoir NULL)
- En cas d'échec : renvoie NULL, et l'ancienne zone reste valide. D'où la variable tmp !



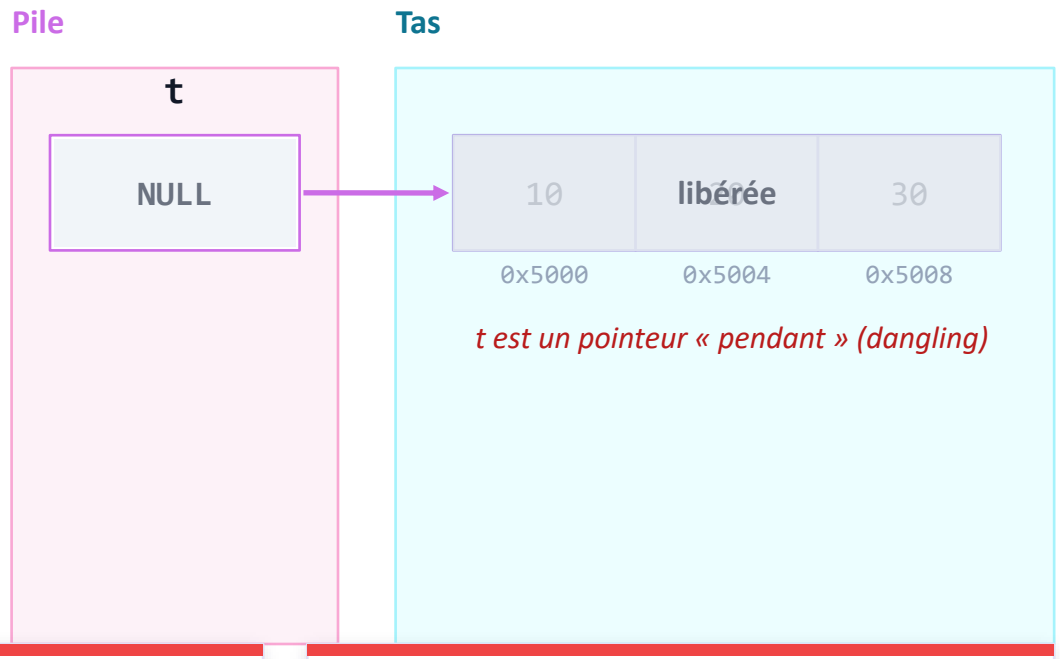
Piège : t = realloc(t, ...); perd l'ancienne zone si realloc échoue : c'est une fuite mémoire.

free : libérer la mémoire

free.c

```c
free(t);      // la zone est rendue au système
t = NULL;     // t ne pointe plus sur rien
```

- p doit avoir été obtenu par malloc, calloc ou realloc
- free(NULL) ne fait rien
- Après free, la zone ne doit plus jamais être utilisée



Fuite mémoire

oublier free : la mémoire n'est jamais rendue

Double libération

appeler free deux fois sur la même zone

Utilisation après libération

lire ou écrire *t après free(t)

Exemple : un tableau de taille variable

moyenne.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Combien de notes ? ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    double *notes = malloc(n * sizeof *notes);
    if (notes == NULL) { perror("malloc"); return 1; }

    double somme = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lf", &notes[i]);
        somme += notes[i];
    }
    printf("Moyenne : %.2f\n", somme / n);

    free(notes);
    return 0;
}
```

1. **Connaître la taille**
   n est lu à l'exécution : impossible avec un tableau statique.

2. **Allouer et vérifier**
   malloc(n * sizeof *notes), puis test de NULL.

3. **Utiliser**
   notes[i] s'utilise exactement comme un tableau.

4. **Libérer**
   Un malloc = un free.

Combien de mémoire peut-on allouer ?

memoire.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t i = 0;
    while (malloc(1024) != NULL)   // 1 Ko à chaque tour
        i++;
    printf("Mémoire disponible : %zu Ko\n", i);
    return 0;
}
```

Sous Linux, attention !

malloc réussit presque toujours (overcommit) : la mémoire n'est vraiment réservée qu'à la première écriture. Le programme peut tourner très longtemps, ou être tué par l'OOM killer.

Une fuite volontaire

Aucun free : toute la mémoire réservée est perdue jusqu'à la fin du processus.

À retenir : un malloc qui ne renvoie pas NULL ne garantit pas que la mémoire physique existe. C'est l'OS qui gère la mémoire virtuelle (CM 01a, rôle 2).

Détecter les erreurs : valgrind et ASan

valgrind

```
$ gcc -g fuite.c -o fuite
$ valgrind --leak-check=full ./fuite
==4242== HEAP SUMMARY:
==4242==   in use at exit: 40 bytes in 1 blocks
==4242== 40 bytes in 1 blocks are definitely lost
==4242==    at 0x48407B4: malloc
==4242==    by 0x109165: main (fuite.c:6)
```

AddressSanitizer

```
$ gcc -g -fsanitize=address prog.c -o prog
$ ./prog
ERROR: AddressSanitizer: heap-use-after-free
```

Bonnes pratiques

1. Tester le retour de malloc / calloc / realloc
2. Un malloc = un free
3. Mettre le pointeur à NULL après free
4. Écrire sizeof *p plutôt que sizeof(type)
5. Compiler avec -g et tester avec valgrind

SYNTHÈSE

À retenir

01

Trois flux

stdin (0), stdout (1), stderr (2). Les erreurs vont sur stderr.

02

getchar renvoie un int

Pour pouvoir distinguer EOF des 256 caractères possibles.

03

Jamais gets

Utilisez fgets(buf, sizeof buf, stdin) : la taille est contrôlée.

04

scanf

Passez les adresses (&n) et vérifiez le nombre de valeurs lues.

05

Le tas

malloc, calloc et realloc réservent ; vérifiez NULL ; realloc via une variable tmp.

06

free

Un malloc = un free, puis p = NULL. Vérifiez avec valgrind.

Ressources pédagogiques

Le manuel, toujours à portée de main

```
$ man 3 printf
$ man 3 scanf
$ man 3 fgets
$ man 3 malloc
$ man valgrind
```

La section 3 du manuel décrit les fonctions de la bibliothèque C. La section 2 décrit les appels système (read, write…), qu'on verra dès le CM 02.

En ligne

cppreference : bibliothèque C (stdio.h, stdlib.h)

Wikilivres : Programmation C

Guide ANSSI : règles de programmation sécurisée en C

Documentation de Valgrind (Quick Start)

lamae

Questions ?

PROCHAINE SÉANCE

CM 02 — Les fichiers

Appels système open, read, write, lseek, close · descripteurs · i-nœuds

Christophe Cruz · christophe.cruz@ube.fr